MediCore Application Report

Project Title: MediCore – AI-Enabled Medical Management System

Version: 1.0.0+1

Submitted to: [Supervisor Name/Course Name]

Author: Nazmus Sakib

Department of Computer Science & Engineering

University Name

Date: November 27, 2025

Abstract

The MediCore application is a **Flutter-based**, cross-platform mobile solution designed to modernize and streamline the medical service delivery process. It addresses the growing need for digital healthcare access by integrating key functionalities, including **online appointment booking**, an **AI-driven symptom checker** powered by the **Google Gemini API**, secure **online payment processing** via **SSLCommerz**, and a comprehensive **medicine store**. Built on the scalable **Firebase** platform, MediCore ensures real-time data synchronization, secure user authentication, and a robust back-end infrastructure. This report details the system architecture, key features, security implementations, and future enhancements, demonstrating MediCore's potential to create a unified and efficient digital health ecosystem for patients, doctors, and administrators.

Contents

Chapter 1

Introduction

1.1 Project Overview

MediCore is a cross-platform mobile application developed using **Flutter** and **Dart**. Its core purpose is to digitalize traditional medical services. The application caters to three primary user roles: **Patients**, **Doctors**, and **Administrators**, providing tailored features for each. The integration of modern mobile technology and Artificial Intelligence is central to the project's design.

1.2 Problem Statement

Traditional healthcare systems often suffer from inefficient processes, including manual appointment scheduling, long wait times, and fragmented medical records. There is a need for a unified digital platform that can offer convenience, real-time access to services, and intelligent health support.

1.3 Scope of the Report

This report will detail the application's architecture, key features, the technology stack employed, the data flow, security measures, and the testing and deployment strategies used to bring MediCore to a functional state.

—

Chapter 2

Objectives

2.1 Primary Goals

- Develop a modern, performant, and intuitive medical management system using the Flutter framework.
- Establish a secure, real-time back-end using **Firebase Authentication** and **Firestore**.
- Implement an **AI-assisted module** for symptom checking and health insights using the Google Gemini API.

2.2 Secondary Goals

- Provide a seamless workflow for online appointment booking and flexible payment integration (SSLCommerz).
- Include an integrated medicine store with robust order management features.
- Build a scalable and maintainable codebase utilizing the **Provider** state management pattern.

—

Chapter 3

Key Features

3.1 User Authentication and Profile Management

- **Multi-factor Authentication:** Standard Email/Password, as well as Google and Facebook Sign-In via Firebase Auth.
- **Role-Based Access Control (RBAC):** Distinct interfaces for Patient, Doctor, and Admin roles.
- User profile creation, update, and secure storage on Firestore.

3.2 Patient-Centric Services

- **Doctor Search and Profile View:** Filterable browsing of doctor profiles with ratings, availability, and specializations.
- **Appointment Management:** Real-time booking, cancellation, and history tracking.
- **AI Symptom Checker:** A conversational interface powered by Gemini to provide preliminary health suggestions.
- **Digital Records:** Secure upload and storage of medical reports and prescriptions using **Firebase Storage**.

3.3 E-Commerce (Medicine Store)

- Comprehensive medicine catalog with detailed product pages.
- Robust Add-to-Cart, checkout process, and order history view.

- **Real-Time Order Tracking** for patients.

3.4 Doctor and Admin Dashboards

- Doctor-specific dashboards for viewing and managing upcoming patient appointments.
- Admin portal for system oversight, user management, and detailed **Order Analytics**.

3.5 Secure Payment Integration

- Integration with the local payment gateway **SSLCommerz**.
- Support for mobile financial services (bKash, Nagad, Rocket) and major cards (VISA/MasterCard).
- Secure, server-side verified payment status updates to Firestore.

3.6 Google Generative AI (Gemini) Features

- **Symptom Analysis:** Users input symptoms and receive health suggestions and recommendations for specialists.
- **AI Chat Interface:** General health and wellness query assistant.

—

Chapter 4

System Architecture

4.1 Conceptual Architecture Model

The MediCore system employs a **three-tier architecture**: Presentation, Application, and Data, built predominantly on a **Serverless Mobile Backend (BaaS)** approach with Firebase.

4.1.1 Technology Stack

Table 4.1: MediCore Technology Stack Summary

Component	**Technology / Service**
Frontend (Mobile)	**Flutter** (Dart)
Backend (BaaS)	**Firebase** (Authentication, Firestore, Storage)
State Management	**Provider**
AI/Generative Model	**Google Gemini API**
Payment Gateway	**SSLCommerz**
Cloud Storage (Assets)	**Cloudinary** (For large assets like profile images)

4.2 Application Structure and State Management

The application follows the common Flutter structure for maintainability and separation of concerns.

4.2.1 Codebase Hierarchy

```
lib/
  main.dart             // Entry point, Firebase/Provider initialization
```

```
app.dart              // MaterialApp, routing, theme setup
config/               // Constants, themes, API keys (.env file handlers)
models/               // Dart classes for data models (User, Doctor, Order, etc.)
providers/            // Application logic, state management (using Provider)
services/             // External API integrations (Auth, Payment, Cloudinary)
ai_engine/            // Dedicated logic for Gemini API calls and processing
screens/              // UI pages, organized by user role
   patient/
   doctor/
   admin/
widgets/              // Reusable UI components
```

4.2.2 Provider State Management

The **Provider** package is used to manage application state, promoting separation of concerns (UI from business logic). Dedicated providers handle specific application domains:

- `AuthProvider` for user session and role.
- `AppointmentProvider` for booking and scheduling.
- `StoreProvider` for cart, catalog, and order management.

—

Chapter 5

Database & Data Flow

5.1 Firestore Database Schema

The application leverages **Firestore**, a NoSQL cloud database, for flexible and real-time data storage. Key top-level collections include:

- `users`: Stores general user data (Patient, Doctor, Admin profiles).
- `doctors`: Specialized collection for doctor profiles, availability, and consultation fees.
- `appointments`: Documents representing booked slots, linked to patient and doctor IDs.
- `orders`: Records of medicine store transactions, including items, total cost, and delivery status.
- `notifications`: Used for push notifications (e.g., appointment reminders, order updates).

5.2 Key Data Flow Diagrams

5.2.1 Appointment Booking Flow

1. Patient browses `doctors` collection.
2. Patient selects slot and initiates payment.
3. Payment processed via SSLCommerz WebView.

4. Upon successful callback, a new document is written to `appointments` with `status: 'Confirmed'`.

5. A notification is generated in `notifications` for both patient and doctor.

5.2.2 AI Query Flow

1. Patient enters symptoms into the chat interface.

2. The app sends the query to the **`ai_engine/`** service.

3. The service calls the **Google Gemini API**.

4. Gemini processes the query and returns a structured response (health suggestions, recommended specialist type).

5. The response is displayed to the user.

—

Chapter 6

Payment Workflow

6.1 SSLCommerz Integration Model

The MediCore payment process follows a crucial two-step transaction verification model to ensure data integrity and prevent fraud.

1. **Initiation:** The app generates a temporary payment record in Firestore (status: 'Pending') and sends a request to the SSLCommerz server with transaction details.

2. **Gateway Handover:** The user is redirected to the SSLCommerz hosted **Web-View** to select the payment method (e.g., bKash, Card) and complete the transaction.

3. **Server Callback:** Upon payment completion, SSLCommerz sends a secure **Instant Payment Notification (IPN)** to the MediCore backend verification server (outside the scope of the app) or directly to a secured Firebase Function/Service.

4. **Status Update:** The verification service authenticates the callback and updates the corresponding record in Firestore (`orders` or `appointments`) to `status: 'Paid'`.

5. **Confirmation:** The app listens for the Firestore update in real-time and displays the confirmation to the user.

—

Chapter 7

Assets and Resources

- **Static Assets:** All images and icons are organized under `assets/images/` and `assets/icons/`.
- **Dynamic Assets:** Lottie animations are used for a modern user experience (e.g., loading screens, success messages) and stored in `assets/animations/`.
- **Configuration:** Sensitive environment keys (API keys, SSLCommerz credentials) are managed in a local `.env` file and accessed securely via environment variables at runtime.
- **Firebase Configuration:** The app-specific configurations for Firebase (e.g., API keys, project IDs) are stored in the generated `firebase_options.dart` file.

—

Chapter 8

Security Measures

8.1 Data Security and Access Control

- **Firestore Security Rules:** Implemented to enforce strict Role-Based Access Control. For example, a Patient can only read/write their own `users` document and only read doctor profiles from the `doctors` collection. Only an Admin can modify the `orders` status.
- **Firebase App Check:** Enabled to verify that incoming requests originate from a legitimate instance of the MediCore application, mitigating unauthorized API usage and denial-of-service attacks.

8.2 Authentication and API Key Management

- **Firebase Authentication:** Handles all user credentials securely, conforming to industry best practices.
- **API Key Secrecy:** Critical keys (Gemini API key, SSLCommerz credentials) are ideally proxied through a secured Firebase Function or Cloud Run service to prevent client-side exposure.
- **Secure Payment Gateway:** SSLCommerz handles all sensitive payment card details, ensuring the MediCore application itself is not required to be PCI compliant.

—

Chapter 9

Testing and Deployment

9.1 Testing Strategy

9.1.1 Unit Testing

- Focuses on the business logic within **Provider** classes (e.g., calculating cart total, appointment logic, data parsing).

9.1.2 Widget Testing

- Verifies that core UI components (`widgets/`) and critical screens render correctly and respond to user interactions as expected.

9.1.3 Integration and Manual Testing

- End-to-end testing of core user flows: Login, Booking and Payment, AI Chatbot, Medicine Ordering.

9.2 Deployment Process

- **Release Build:** Generated using the Flutter release command.
- **Optimization:** **Code Minification** and **obfuscation** are enabled to protect intellectual property and reduce the final application size.
- **Target Platforms:** Primary deployment on **Android** (Google Play Store), with optional verification for **iOS** and **Web**.

—

Chapter 10

Challenges and Solutions

- **Challenge: Secure Payment Verification**
  - **Solution:** Implemented server-side verification callback logic (e.g., via a Firebase Function) to update the payment status securely.
- **Challenge: Firestore Performance and Cost Management**
  - **Solution:** Optimized data querying via indexing and minimized reads/writes by using Cloudinary for large asset storage.
- **Challenge: Optimizing AI Responses**
  - **Solution:** Applied careful prompt engineering to the Gemini API calls to ensure concise and relevant health suggestions with appropriate disclaimers.

—

Chapter 11

Future Enhancements

- **Telemedicine Integration:** Live **doctor video consultation** feature (e.g., WebRTC integration).
- **EHR and Prescription Generation:** Module for doctors to generate secure digital prescriptions and manage Electronic Health Records (EHR).
- **Dedicated Admin Dashboard:** Creation of a full-featured web-based administrator portal.
- **Wearable Device Integration:** Connect with health tracking devices (e.g., Apple Health Kit, Google Fit).

—

Chapter 12

Conclusion

The MediCore application successfully integrates modern mobile development (Flutter), a robust serverless backend (Firebase), a secured local payment gateway (SSLCommerz), and cutting-edge Generative AI (Google Gemini) to deliver a comprehensive digital health ecosystem. The system is architecturally sound, scalable, and built with a strong emphasis on user experience and security. MediCore fulfills its primary objective of simplifying healthcare access and stands as a strong foundation for future telemedicine and health management innovations.

Bibliography

[1] Flutter Team. Flutter Documentation. Available at: https://docs.flutter.dev/

[2] Google. Firebase Documentation. Available at: https://firebase.google.com/docs

[3] Google. Google Generative AI Documentation (Gemini API). Available at: https://ai.google.dev/

[4] SSLCommerz. Developer Documentation. Available at: https://developer.sslcommerz.com/

[5] Remi Rousselet. The Provider package. Available at: https://pub.dev/packages/provider